ZhuLong: An Execution-Grounded LLM Agent for EDA Scripting

Anonymous Author(s)

Abstract

EDA scripting with tool-specific, often undocumented APIs remains a long-tail bottleneck that existing LLM-based tooling fails to address. This paper presents **ZhuLong**, an execution-grounded LLM coding agent for PyAether, SKILL, and Tcl that combines API retrieval, documentation inspection, and sandbox execution via unified MCP tools into a closed generate–execute–refine loop. We further provide a **theoretical grounding** for why runtime observation is necessary: EDA correctness is a *situated* property that cannot be guaranteed from static knowledge, and we formalize this "grounding gap" with a Bayesian argument showing it is bounded by observation fidelity—not model scale. The Harness supplies precisely that observation, by reading the environment's documented API facts and by reading back what executing code does to the live design.

We evaluate ZhuLong on **EDA-Eval-PyAether**, a benchmark of 158 real-world tasks with assertion-based execution, where the complete system achieves **81.4%** Pass@1 ($\pm$1.5, 5-run mean) in the commercial Empyrean Aether environment, substantially outperforming a pure LLM baseline (**10.5%**). Ablation studies show the execution loop is decisive: adding sandbox execution to retrieval yields +9.2 pp, while removing either execution ($-$9.2 pp) or retrieval ($-$0.4 pp) reduces accuracy—consistent with the theoretical prediction. The main quantitative results are measured on the 158-task PyAether slice; a cross-language reproduction of the component ablation on the SKILL (158) and Tcl (133) slices of the unified **EDA-Scripting-Bench** (449 tasks total) reproduces the same structure—the complete system is the top and the prior-only baseline the bottom on every language—and the fidelity (Ω) and belief-update (Φ) axes are reported on PyAether only.

CCS Concepts

• **Hardware** → **Software tools for EDA**.

Keywords

LLM Agents for EDA, EDA Scripting, Code Generation, Sandbox Execution, Execution Grounding

ACM Reference Format:
Anonymous Author(s). 2026. ZhuLong: An Execution-Grounded LLM Agent for EDA Scripting. In *Proceedings of* . ACM, New York, NY, USA, 13 pages.

1 Introduction

Large language models (LLMs) have demonstrated strong capabilities in general-purpose code generation [17, 23, 25]. However, applying LLMs to EDA scripting—where engineers routinely manipulate in-memory design databases, create layout instances, route schematics, and query netlists—remains challenging. First, EDA APIs are long-tailed and tool-specific; many rarely appear in public training corpora. Second, correctness cannot be verified statically: it depends on observable side effects in a real EDA environment, such as modified layouts or updated schematics. Third, without execution feedback, static generation cannot diagnose errors or iteratively refine incorrect code [3, 5]. Existing LLM-based EDA work has mainly focused on HDL generation [15, 21], while scripting with real tool APIs—especially with incomplete documentation and sandbox execution—remains underexplored.

This paper presents **ZhuLong**, an execution-grounded LLM coding agent for PyAether, SKILL, and Tcl. Named after a mythical creature whose opened eyes bring light, ZhuLong aims to illuminate opaque EDA APIs and automate tool-specific scripting workflows. Built on Cline-CLI [7], ZhuLong extends three MCP tools [16]—`search_apis`, `get_api_details`, and `run_code`—that form a closed loop: retrieve candidate APIs, inspect their documentation, execute code in the EDA environment with execution feedback, and iteratively refine based on observed errors or outputs. All tools accept a unified `lang` parameter, so the same agent logic covers PyAether, SKILL, and Tcl. Beyond the engineering, we provide a **theoretical foundation** (Section 4): EDA scripting correctness is a *situated* property that cannot be guaranteed from static knowledge, and a Bayesian analysis shows the resulting "grounding gap" is bounded by observation fidelity—not model scale—explaining why runtime observation is essential, and why the Harness must supply both halves of it: reading the environment's documented API facts, and reading back what executing code does to the live design.

The main contributions are:

- **An account of *which* factor sets the ceiling, with measurements that distinguish them (RQ1).** We formalize the *grounding gap*: EDA correctness is a situated (L4) property that no offline knowledge can guarantee, and a Bayesian argument shows the resulting gap is bounded by *observation fidelity* rather than by the capacity of the model (Section 4). We answer RQ1 from both sides—laddering the observation channel with the LLM fixed, and changing the prior with the harness fixed—and the framework is stated *before* the measurements, so that its claims are predictions the experiments could refute rather than explanations fitted afterwards.
- **A harness whose necessary components we isolate (RQ2).** ZhuLong couples three unified MCP tools—`search_apis`, `get_api_details`, and `run_code`—so that API retrieval and documentation inspection

,

(the *observation channel*), sandbox execution (the *effector*), and the generate–execute–refine loop (the *belief update*) form one closed loop, driven by a single `lang` parameter so the same agent logic covers PyAether, SKILL, and Tcl (Section 3). In a commercial Empyrean Aether environment, ZhuLong attains **81.4%** Pass@1 on the 158-task PyAether slice, versus **10.5%** for a pure LLM and **72.2%** for a retrieval-augmented static baseline. The same component ordering reproduces across the SKILL and Tcl slices of EDA-Scripting-Bench (Section 6.2.4). Ablation shows that the three components are *jointly necessary and not interchangeable*: adding sandbox execution to RAG yields **+9.2 pp**, while removing execution (−9.2 pp) or retrieval (−0.4 pp) reduces accuracy, and bounding the belief update degrades it monotonically (Section 6).

- **A cross-language benchmark on which the same structure can be re-measured per toolchain (RQ3). EDA-Scripting-Bench** places PyAether, SKILL and Tcl under one task schema, one assertion criterion and one harness, so that the structure of RQ1–RQ2 can be re-measured slice by slice with nothing changed but the toolchain (Section 5). Its completed PyAether slice, **EDA-Eval-PyAether** (158 tasks from four sources, expert-reviewed), serves as the first standardized PyAether benchmark; the SKILL and Tcl slices complete the suite for reproducible comparison of LLM-based scripting agents. What we release is the **harness, the task schema and the evaluation protocol**; the task cases themselves derive from vendor documentation and customer designs and cannot be redistributed.

2 Background and Motivation

We target two widely-used EDA scripting interfaces: PyAether for Empyrean Aether [9] and SKILL for Cadence Virtuoso [20]. Despite their prevalence, scripting for both environments poses challenges beyond what generic LLM code generators can address, and the community lacks basic infrastructure to measure progress.

EDA scripting is fundamentally different. Prior work on LLM code generation has succeeded in domains where correctness is verified by unit tests [2, 6]. EDA scripting presents three distinct challenges: (i) APIs are long-tailed, tool-specific, and poorly documented; (ii) correctness depends on side effects in a live, stateful database, not return values; and (iii) no benchmark exists to evaluate whether generated scripts actually work.

Prior work stops short of execution. Recent efforts like LayoutCopilot [14], ChaTCL [19], and RAG-EDA [18] combine LLMs with static knowledge bases. Yet they all share a critical limitation: they do not execute generated code. Correctness is assessed by human judgment [14, 19] or retrieval quality [18], not by whether the script actually modifies a layout. Consequently, errors go undiagnosed, and the gap between "looks plausible" and "actually works" remains unbridged.

The root problem: no benchmark, no progress. Without a standardized benchmark, the community cannot systematically compare approaches or measure progress—each prior work uses ad-hoc tasks with different protocols. Recent concurrent benchmarks for Tcl/Innovus flows [13, 24] confirm the growing recognition of this gap, yet target different toolchains. A dedicated PyAether/SKILL benchmark remains absent.

Our approach. ZhuLong addresses the technical challenges through sandbox execution feedback—closing the loop from generation to execution to refinement—together with a **theoretical grounding** (Section 4) that explains *why* execution is necessary. Crucially, we introduce **EDA-Eval-PyAether**, the first standardized benchmark for PyAether scripting. Together, these establish the methodology, the theory, and the measurement infrastructure for advancing LLM-based EDA scripting. The contribution boundary of this paper is the *harness*—the three MCP tools (`search_apis`, `get_api_details`, `run_code`), the evaluation protocol, and the benchmark; the *agent backbone* (the underlying model and its training) is outside the scope of this paper.

3 System Design

3.1 Overall Architecture

ZhuLong is built on Cline-CLI [7], an open-source framework for autonomous coding agents, extended with EDA-specific retrieval, documentation, and execution capabilities. Throughout, we call the resulting runtime an **ICL-type autonomous agent**: an agent that decides its own next action, keeps no explicit posterior, and maintains its belief state implicitly by in-context learning, as formalized in Section 4. We choose Cline-CLI for its mature permission controls, human-in-the-loop support, and enterprise readiness. As shown in Figure 1, the system comprises three components: the LLM-based agent runtime, the API knowledge base, and the EDA execution environment. The agent interacts with these components through three MCP tools.

3.2 Three MCP Tools

ZhuLong extends three MCP tools [1], each accepting a unified `lang` parameter (`pyaether`, `skill`, or `tcl`) to support all three platforms with identical agent logic:

- `search_apis(query, lang)` retrieves relevant APIs via semantic and keyword-based search. We index APIs by name and description using BGE-M3 embeddings [4] with FAISS [8] for efficient similarity search, returning the top-K results ($K = 5$ by default) with confidence scores and descriptions.
- `get_api_details(api_name, lang)` fetches comprehensive documentation for a specified API, including parameter types, return values, and examples.
- `run_code(code, mode, lang)` executes the generated code in the EDA environment. It supports two modes: (1) **lightweight**—runs in a sandboxed environment

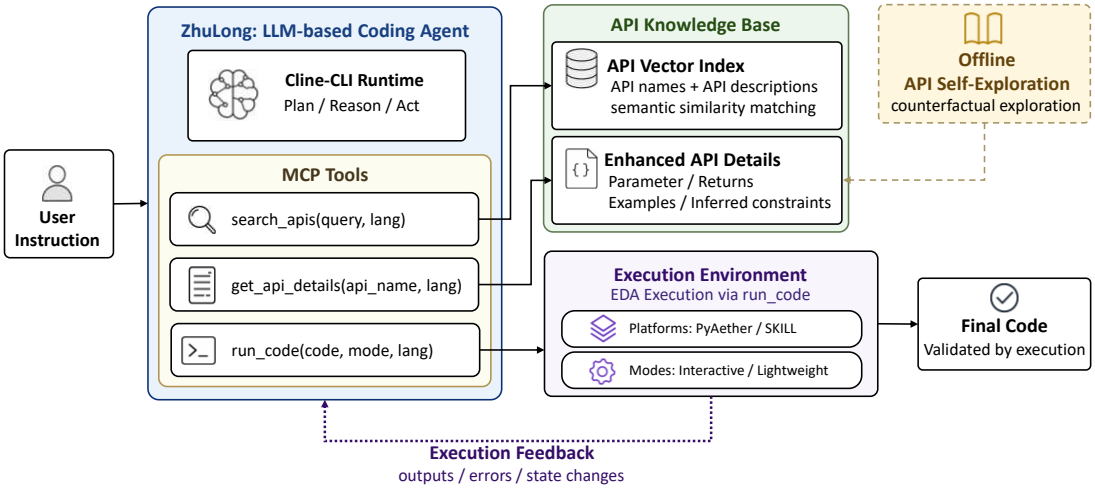


Figure 1: ZhuLong system architecture. The agent interacts with API retrieval, documentation, and EDA execution via three unified MCP tools. Execution feedback supports iterative refinement.

without a GUI session (standalone PythonE for PyAether, or SKILL in batch mode for Virtuoso), capturing outputs, errors, and state changes for iterative refinement; (2) **interactive**—runs inside an open EDA session via the agent-EDA bridge (socket communication), enabling operations on unsaved layouts and schematics.

3.3 Agent Workflow

The agent iteratively refines its output through execution feedback [7]. A typical trajectory begins with reasoning over the user instruction, followed by API retrieval via `search_apis` when needed. Candidate APIs are then inspected via `get_api_details`, and code is generated accordingly. The agent executes the code via `run_code`, observes the result, and re-plans—revising the code, retrieving alternative APIs, or terminating. This loop continues until success or the agent determines further attempts are unlikely to succeed. The agent is not constrained to a fixed sequence; it may skip retrieval when it already knows the required API, or conduct multiple rounds of retrieval when initial results lack confidence.

4 Theoretical Grounding: The Grounding Gap

Our empirical results show that the execution loop is decisive—adding sandbox execution to retrieval yields +9.2 pp (McNemar $p<10^{-8}$), while removing either execution (−9.2 pp, $p<10^{-8}$) or retrieval (−0.4 pp, $p=0.81$, not significant) reduces accuracy. The retrieval-removal gap is not statistically significant on the PyAether-only slice, which actually strengthens the "execution > retrieval" narrative: the effector, not the documentation channel, is what the correctness criterion depends on. This section explains *why* this is not an accident of our implementation but a structural property of EDA scripting. We condense the *grounding gap* framework: EDA scripting correctness is a *situated* property that cannot be guaranteed from offline knowledge, and execution is the only channel that observes the current design state. The gap is bounded by observation fidelity, not model scale.

4.1 A Four-Level Hierarchy of Information

To make precise what can be learned offline versus what must be observed online, we distinguish four levels of information (Table 1). The distinction applies to any agent whose knowledge is fixed by offline learning before deployment:

- **L1–Syntactic Correctness**: well-formedness of symbols according to formal rules. Captured by formal grammars and the syntactic capacities of modern LLMs.
- **L2–Physical Plausibility**: what is physically plausible or generally expected—objects do not pass through walls, water flows downhill. L2 captures robust statistical patterns of the world, not logical certainties; these regularities are general and acquirable offline, constituting the agent's *prior* over how the world typically behaves.
- **L3–Historical Factuality**: what has actually occurred or is documented as fact. L3 concerns actual rather than merely possible states of affairs. Like L1 and L2 it can be acquired offline, but it is *past-oriented*: it describes what was true as of the training cutoff, with no connection to what is true *now*.
- **L4–Situated Truthfulness**: what is true *right now*, in this specific environment. Three characteristics define it: *indexicality*—truth depends on context and changes over time; *non-guaranteeability from offline data*—no

offline dataset can *guarantee* the current state, though it may induce strong priors; and *real-time commitment*—L4 must be acquired continuously and acted upon promptly.

Two clarifications matter. First, L4 is not a semantic category of propositions; it is a *grounding requirement* imposed by indexical reference—the same proposition may be L3 at one moment and L4 at another, depending on whether it refers to a past fact or a current state. Second, the four levels are not a taxonomy of all knowledge: they separate what can be *known in advance* from what must be *observed now*. L4 is whatever must be *read* from the environment rather than *recalled* from memory.

For EDA scripting, the situated state is the live design database: which cell and view are selected, and what the layout or schematic contains at this moment. Situated states are not limited to the physical world—they include digital and informational states generally, and the user's current intent—so a clarification obtained from the user is an L4 observation as well.

The correctness criterion we adopt—that generated code executes without error and all assertions over the mutated design pass—is an L4 property. A script that is well-formed (L1), plausible (L2), and invokes documented functions (L3) can nonetheless violate an undocumented binding contract, depend on an internal selection state, or target the wrong cell, and thus fail the test oracle. No amount of offline knowledge can resolve such failures; they must be *observed* by acting on the live design and reading back the result. Four architectural consequences follow from L4's defining characteristics: a system trained offline and deployed frozen has no observation modules, no persistent situated state, no execution interface, and no real-time feedback loop. Every failure we analyze in Section 6.3 traces to one or more of these.

4.2 A Bayesian Formalization of the Grounding Gap

We cast grounding as decision-making under partial observability, mirroring POMDPs and Bayesian filtering [11, 22]; the formulation below is diagnostic rather than algorithmic—it is not a blueprint for tractable inference in high-dimensional state spaces, but a normative standard that makes precise *what* is missing: the observation likelihood Ω and the recursive update Φ.

Let Θ denote the space of L4 states (the live EDA design database), o_t a runtime observation, a_t a generated code action, and $\mathcal{K}$ the offline knowledge encoded in the LLM's frozen parameters (L1–L3). The agent maintains a belief over the current L4 state:

$$b_t(\theta) \triangleq p(\theta_t \mid o_{1:t},\, a_{1:t-1},\, \mathcal{K}), \tag{1}$$

updated recursively by Bayes' rule:

$$b_t(\theta) \propto p(o_t \mid \theta_t) \int_{\Theta} p(\theta_t \mid \theta_{t-1}, a_{t-1})\, b_{t-1}(\theta_{t-1})\, d\theta_{t-1}, \tag{2}$$

where $p(o_t \mid \theta_t)$ is the *observation likelihood* Ω, implemented by the Harness's observation modules (Section 4.3), and $p(\theta_t \mid \theta_{t-1}, a_{t-1})$ is the *transition model* $\mathcal{T}$ supplied by the LLM's prior over how EDA operations transform the design. Actions are chosen by minimizing expected cost:

$$a_t^* = \arg\min_{a \in \mathcal{A}} \mathbb{E}_{\theta_t \sim b_t}\big[\mathcal{C}(\theta_t, a, G) + \gamma\, V(b_{t+1})\big], \tag{3}$$

where $\mathcal{C}$ encodes the task assertions and γ a discount factor.

The *grounding gap* is the divergence between the posterior achievable with runtime observations and the posterior from the offline prior alone:

$$D_{\text{grounding}}(t) \triangleq D_{\text{KL}}(p(\cdot \mid o_{1:t},\, \mathcal{K},\, \text{env}) \,\|\, p_{\mathcal{K}}(\cdot \mid o_{1:t}))\,. \tag{4}$$

This divergence is bounded by the fidelity of the observation channel, not by the capacity of the model. If $I(\Theta_t; O_t \mid \mathcal{K}) = 0$—that is, the observation channel is silent about the state—no improvement in the prior can recover the current state. Scaling $\mathcal{K}$ improves L1–L3, but it cannot reduce the gap below the floor imposed by Ω.

The source of the gap is causal, not statistical. Parameters are fixed before deployment; facts that come into existence after the training cutoff, or states that depend on unrepeatable circumstances, cannot be present in them by definition. The environment at time t is not a function of the training corpus, and offline data and scaling affect only the prior, never the availability of runtime observation.

This separates grounding from learning-based adaptation. Fine-tuning and meta-learning are capabilities acquired during *training*: they improve the *prior* over L1–L3 distributions. Grounding occurs during *deployment* and requires the likelihood and a posterior update from observing o_t. A better prior does not eliminate the need for observation; the two are complementary—one offline, one online; one about knowledge, one about belief.

4.3 ZhuLong as an Instance of the Agent Harness

The formulation maps onto ZhuLong's architecture. The frozen LLM supplies the prior $\mathcal{K}$ (L1–L3), the transition model $\mathcal{T}$, and an approximation to the policy Π. The Harness supplies what the LLM lacks—observation, belief update, and execution—and ZhuLong's three MCP tools instantiate it: `search_apis` and `get_api_details` are the *observation modules* Ω, reading documented API facts out of the environment; `run_code` is the *effector* that acts on the live design database, whose resulting state is transduced into textual observations (stdout, errors, assertion outcomes). The replanning loop (Section 3.3) implements the belief update Φ: each observation o_t revises the agent's belief about which API contracts and compositions actually hold. The closed loop $b_t \to a_t \to o_{t+1} \to b_{t+1}$ is what turns one-shot generation into grounded search; grounding is a property of the coupled system, not of the model alone.

4.4 Design Principles

The formalization yields three architectural principles: *freeze* $\mathcal{K}$, *maximize* the fidelity of Ω, and *run* Φ *continuously*.

Table 1: The four levels of information

Level	Name	Core Question	Learnable Offline?
L1	Syntactic Correctness	Is this string well-formed?	Yes
L2	Physical Plausibility	Could this event happen?	Yes
L3	Historical Factuality	Did this event happen?	Yes
L4	Situated Truthfulness	Is this true *right now*?	**No**

Decouple knowledge from belief. Knowledge $\mathcal{K}$ is the static L1–L3 prior encoded in the LLM's parameters—general and context-free; belief $b_t(\theta)$ is the dynamic posterior maintained by the Harness—specific and context-dependent. Updating $\mathcal{K}$ with the limited observations available during deployment is inefficient and risks catastrophic forgetting, and even a prior enriched offline cannot substitute for observation; freezing $\mathcal{K}$ and absorbing adaptation in b_t yields stability and flexibility at once.

Optimize observation over the prior. The gap is bounded by the fidelity of the likelihood Ω, not by the expressiveness of the prior: $D_{\text{grounding}}(t)$ is bounded by $\|\tilde{p} - p_{\text{real}}\|$, a bound independent of model scale. Investment in observation—retrieval and documentation quality, execution read-back, state estimation—therefore has a direct and bounded effect, whereas investment in model scale has diminishing returns once the prior is sufficiently rich, because it cannot push the gap below the floor imposed by observation.

Design for continuous belief updating. Φ operates online: every observation yields a recursive update $b_t \mapsto b_{t+1}$. Two implications follow. First, Φ's update rate must exceed the timescale of L4 dynamics, otherwise b_t goes stale. Second, b_t must be a structured representation that captures uncertainty and history, so that the agent can recover from unexpected changes. In the ICL-type autonomous agent of Section 3, Φ has no explicit posterior: it is approximated by in-context learning, since each returned observation is appended and conditions the next decision, while the agent decides for itself when the result is good enough to submit. Because such an agent is autonomous, the quantity to bound is not a count of "rounds"—it has none—but how much evidence its belief state may ever accumulate: a graded budget on the executions it may observe, or a read-back that lags the action it reports.

These principles *account for* our ablation findings rather than merely restating them. Removing the observation channel ("ZhuLong w/o Sandbox") leaves b_t uninformative: with no Ω, L4 violations such as hidden type contracts and internal selection state (Section 6.3) are unobservable by construction, and Pass@1 drops by 9.2 pp. A retrieval-only system (RAG, 72.2%) improves the L1–L3 prior but still cannot observe L4, so its ceiling is bounded by what offline knowledge can guarantee.

Three diagnostic questions follow for any candidate system: (1) does it implement Ω with sufficient fidelity to track L4 dynamics? (2) does it maintain b_t as a structured state, updated through Φ after each observation? (3) does it approximate Π from offline knowledge and execute a_t through a causal interface? Section 6 answers them for ZhuLong, on the benchmark constructed in Section 5.

These principles and questions organize the evaluation into three research questions (Section 6.1): whether the ceiling is set by observation or by the prior (RQ1), which components are necessary and whether they are interchangeable (RQ2), and whether the structure follows from the domain class rather than from one toolchain (RQ3).

5 Benchmark Construction

5.1 Dataset: EDA-Scripting-Bench

For DAC, we assemble **EDA-Scripting-Bench**, a unified cross-language benchmark spanning three EDA scripting languages—PyAether, SKILL, and Tcl—under one task schema and one execution-based protocol. Every task carries a `lang` field and language-appropriate assertions, so a single agent harness (Section 3.3) can be scored uniformly across platforms. EDA-Scripting-Bench currently comprises three completed slices: **EDA-Eval-PyAether** (158 PyAether tasks), a 157-task SKILL slice, and a 134-task Tcl slice (aggregating the Custom WaveView and Innovus runtimes, 77 and 57 tasks respectively), for **449 tasks in total**. The main quantitative results of this paper are measured on the PyAether slice; a cross-language reproduction of the component ablation on the SKILL and Tcl slices is reported in Section 6.2.4, and the fidelity (Ω) and belief-update (Φ) axes are reported on PyAether only.

5.2 Dataset: EDA-Eval-PyAether (PyAether slice)

To evaluate LLM-based code generation for PyAether, we constructed **EDA-Eval-PyAether**, the 158-task PyAether slice of EDA-Scripting-Bench. Tasks were collected from four sources: API references (61.4%), documentation examples (27.2%), internal training materials (7.6%), and anonymized CAD practical cases (3.8%). Each task underwent manual review by EDA domain experts.

Table 2 shows the distribution of tasks across different EDA scenarios. Layout and Schematic operations constitute the majority (70.2%), reflecting the core focus of PyAether usage in physical design workflows.

Table 2: Task distribution by scenario

Scenario	Tasks	Percentage
Layout	64	40.5%
Schematic	47	29.7%
Design Management	24	15.2%
Others	23	14.6%
Total	158	100%

5.3 Evaluation Protocol

Each task consists of a natural language prompt, a function signature, and assertion-based test code. A task is considered passed if the generated code executes in the PyAether sandbox without errors and all assertions pass. The primary metric is Pass@1 [6]:

$$\text{Pass@1} = \frac{\#\text{ tasks with all assertions passed}}{\text{Total tasks}} \times 100\%$$

6 Experiments

6.1 Experimental Setup

6.1.1 Research Questions. The framework of Section 4 makes three testable claims: that the grounding gap is bounded by observation fidelity rather than by the prior; that bridging it requires a runtime architecture supplying observation, belief update and execution; and that this follows from the domain's structure rather than from one toolchain. We ask one research question per claim.

- **RQ1 (Ceiling: observation or prior?).** Is the achievable accuracy bounded by the fidelity of the observation channel Ω, or by the strength of the frozen prior $\mathcal{K}$? We answer from both sides: the observation-fidelity ladders (S1) vary Ω with the LLM fixed, and the prior-axis contrast (S3) varies $\mathcal{K}$ with the harness fixed.
- **RQ2 (Which components are necessary, and are they interchangeable?).** Does bridging the gap require observation, execution through a causal interface, and continuous belief update—and can one of the three substitute for another? S2 varies the effector's presence and the belief-update budget.
- **RQ3 (Domain class or toolchain?).** Does the same structure reproduce on a structurally different toolchain? The protocol of S1–S2 is re-run on the SKILL and Tcl slices of **EDA-Scripting-Bench**, so the comparison is made *within* one benchmark rather than across separate artifacts.

RQ1 and RQ2 test the framework's structural claims; RQ3 tests whether they follow from the domain's structure rather than from one toolchain. Because RQ1's two arms sit in different studies, we read them against each other rather than in isolation.

6.1.2 Benchmark and Evaluation Protocol. We evaluate ZhuLong on **EDA-Eval-PyAether**, 158 real-world PyAether scripting tasks, using **Pass@1** as the primary metric. Each task runs in a single trace with per-execution timeout 2500 s; the agent may iterate within a trace via Φ but is never restarted. A file-based `PreToolUse` hook enforces a tool whitelist, blocks shell/code-search, and confines file access to a fresh empty workspace, so neither task assertions nor solution copies are reachable; the hook is identical across all arms (details in Appendix). Each configuration is evaluated over five independent runs with reshuffled task order; accuracy is mean±std. Trace-aggregated quantities (read-back counts, tool-call totals) are single pooled values; effect sizes (Δ) use table means. Generation-stage failures (no executable code produced) are reported separately from execution-stage failures. Runs suffering infrastructure faults (license exhaustion, port unreachable, insufficient disk) are discarded and re-run. The full-system anchor must reach $\geq$75% Pass@1 as a health floor; ablation arms have no such floor. All figures are on the PyAether slice; cross-language results in Section 6.2.4.

6.1.3 Baselines and ZhuLong Variants. To isolate each component's contribution, we use two baselines and two ablations (Table 3). Retrieval and documentation form the observation channel Ω; the sandbox is the effector. Each capability is toggled by a tool-registration switch—a single disabled-list in the server environment spanning the four core tools—so a disabled tool vanishes from the agent's tool set while every enabled tool keeps a byte-identical implementation across all arms.

These columns name components realized at the harness boundary. The sandbox is removed by unregistering `run_code`; retrieval by unregistering the three retrieval tools; the anti-leak hook stays loaded in every arm.

Table 3: Configurations and their components

Configuration	Retrieval	Sandbox
Baseline 1 (Pure LLM)	✗	✗
Baseline 2 (RAG)	✓	✗
ZhuLong w/o Retrieval	✗	✓
ZhuLong (Full)	✓	✓

6.1.4 LLM Backbones. All main experiments use DeepSeek-V4-Pro as the primary backbone. To understand the impact of different LLMs, we additionally evaluate GLM-5.2, DeepSeek-V4-Flash, Kimi-K2.6, and Doubao-Seed-2.0-Pro.

6.2 Results and Ablations

The framework of Section 4 fixes what each study must vary. Table 4 gives the progressive build-up and the component ablation as an overview; the studies below then change one component at a time, with the LLM frozen unless stated otherwise.

6.2.1 S1: Observation fidelity (RQ1). Principle 2 predicts that with the model fixed, accuracy is bounded by the fidelity of the observation channel Ω. We vary Ω along the two axes

Table 4: Ablation study results

Configuration	Pass@1 (%)	Δ
Progressive build-up (vs. previous row)		
Baseline 1: Pure LLM	10.5 ± 1.9	—
Baseline 2: RAG	72.2 ± 1.6	+61.7 pp
ZhuLong (Full)	**81.4 ± 1.5**	**+9.2 pp**
Component ablation (vs. ZhuLong Full)		
ZhuLong w/o Sandbox	72.2 ± 1.6	−9.2 pp***
ZhuLong w/o Self-Expl.	77.3 ± 2.9	−4.1 pp*
ZhuLong w/o Retrieval	81.0 ± 4.5	−0.4 pp^{ns}

Significance from McNemar test on pooled per-task pass/fail across 5 runs (790 paired observations per arm) and paired bootstrap (10 000 resamples, 95% percentile CI): ***p¡0.001, **p¡0.01, *p¡0.05, ns = not significant. w/o Sandbox: $p<10^{-8}$, CI $[-13.1, -5.4]$ pp. w/o Self-Expl.: $p<0.05$, CI $[-7.0, -1.2]$ pp. w/o Retrieval: $p=0.81$, CI $[-3.5, +2.9]$ pp (not significant).

through which it receives information, holding the LLM fixed in both.

Documentation axis. Table 5 holds the agent loop and the LLM fixed, and changes the documentation channel together with the presence of the effector: no observation at all; a low-fidelity index built from API names alone, coupled with the full execution loop; and a high-fidelity index that adds descriptions, BM25 term matching, abbreviation expansion and synonym redirect, used without an effector as the RAG baseline.

Table 5: Documentation fidelity with and without the effector (LLM and loop fixed; (L) includes the effector, (H) is the retrieval-only RAG baseline without it)

Rung	Configuration	Pass@1 (%)
(N) No observation	Baseline 1 (Pure LLM)	10.5 ± 1.9
(L) Low fidelity	name-only index	82.8 ± 1.0
(H) High fidelity	+ description, BM25, abbreviations, synonyms	72.2 ± 1.6

The documentation axis is non-monotone: (L) at 82.8% exceeds (H) at 72.2%. This reflects the effector, not the index—(L) couples a name-only index with the full execution loop, whereas (H) is retrieval-only without execution. A coarse index with execution outperforms a rich index used only for static retrieval.

Read-back axis. We compare three closed-loop variants differing only in how much live state execution returns: (N′) *no observation*—prior only; (B) *binary feedback*—`run_code` reports only pass/fail; (F) *full observation*—stdout, exception traces, per-assertion outcomes, and queried state (the default). The three rungs compress the *same* executed state to different bandwidths: (F) returns the full `error_log/print_log/result`; (B) collapses it to one bit (`PASS` = no exception, not that assertions held); (N′) returns only `executed`. Principle 2 predicts a monotone gain (N′)<(B)<(F).

Table 6 reports Pass@1 and the mean number of execution read-backs per variant.

Table 6: Observation-fidelity ablation along the read-back axis (RQ: Principle 2)

Observation channel (Ω)	Pass@1 (%)	Mean read-backs
(N′) None (prior only)	73.3 ± 2.2	—†
(B) Binary pass/fail	78.1 ± 1.3	22.3
(F) Full observation	81.4 ± 1.5	12.1

The results recover the predicted monotone ordering: (N′) $73.3 \pm 2.2\% <$ (B) $78.1 \pm 1.3\% <$ (F) $81.4 \pm 1.5\%$. N′ → B (+4.8 pp) is the gain from learning whether a run passes at all; B → F (+3.3 pp) is the further gain from seeing the concrete error that turns a categorical verdict into actionable evidence.[1]

Beyond accuracy, the read-back rungs differ sharply in *convergence cost* (Table 7). Dropping from (F) to (B) inflates mean `run_code` calls from 12.1 to 22.3 (∼1.8×), wall-clock from 2.22 h to 3.73 h (∼1.7×), and timed-out tasks from 3.4 to 7.6 (∼2.2×): under (B) the agent learns only pass/fail with empty error logs, so it must hypothesize corrections blind and re-execute—one (B) trace issues 287 `run_code` calls on a single task. This is the cost-side counterpart of the accuracy result: poorer observation inflates the search effort needed to converge, exactly the "fidelity down → convergence cost up" trade-off Principle 2 implies.

Table 7: Convergence cost along the read-back axis (LLM and effector fixed). (N′) cost is not recoverable (traces discarded in a storage migration); comparison restricted to (F) vs. (B).

Read-back	Pass@1 (%)	Mean `run_code`	Wall (h)	Timeout
(F) Full	81.4 ± 1.5	12.1	2.22	3.4*
(B) Binary	78.1 ± 1.3	22.3 ± 3.3	3.73 ± 0.72	7.6 ± 3.4
(N′) None	73.3 ± 2.2	not recoverable†		

* mean of 5 runs (range 1–6). † (N′) traces discarded; cost not imputed. (B) columns are 5-run means ± sample std.

6.2.2 S2: The closed loop (RQ2). Principle 3 requires two things: that actions reach the design through a causal interface, and that the belief update Φ run continuously.

Effector: does the action reach the live design? Removing the sandbox drops Pass@1 from 81.4% to 72.2% (−9.2 pp; McNemar $p<10^{-8}$, 95% CI $[-13.1, -5.4]$ pp); adding it to the RAG baseline raises it by the same amount. Without an effector, b_t receives no evidence about the live state and the agent is reduced to a prior-only predictor. Execution returns concrete error messages that guide corrections, transforming one-shot generation into iterative search [10, 12].

[1] An earlier legacy probe of the (B) arm showed an N′-over-B inversion (66.2±16.1%, one anomalous run at 43.0%); the fresh 5-run re-measurement collapses the spread to ±1.3 and restores the predicted ordering, confirming the dip was an instability artifact.

Belief update: how much evidence may Φ accumulate? ZhuLong is autonomous: the agent decides when to submit, so we do not shorten the loop or force early submission. Instead we bound how often the loop *may execute*—the number of `run_code` calls per trace—which determines how much state evidence b_t receives. Once the budget is spent, the call is denied at the tool boundary; the agent keeps running under its own control until it submits. Table 8 also includes a *lagged* channel: a single-slot (L=1) buffer whose read-back returns the state from the *previous* execution, leaving the agent permanently one action behind the design. Both axes leave the LLM, observation content, and effector untouched. Principle 3 predicts accuracy rises with the execution budget but saturates once updates outpace the design's dynamics.

Table 8: Effect of bounding the evidence available to Φ (LLM, Ω and effector fixed). k = max `run_code` calls per trace; "lagged" = read-back returns the *previous* execution's state. "Converged" = share of traces where the agent self-terminated.

Update budget	Pass@1 (%)	Converged (%)	Mean gen-fail
Φ k=1	63.8 ± 3.2***	76.3	34.6
Φ k=3	69.1 ± 2.5***	80.4	27.6
Φ k=10	77.7 ± 2.2**	90.4	17.4
Φ lagged	73.0 ± 3.2***	97.5	9.0
Φ unbounded (default)	81.4 ± 1.5	99.1	10.4

5-run means (Pass@1 ± std, n=5); unbounded anchor = full-system baseline (Table 4). *Mean gen-fail* = per-run count of tasks producing no executable code. Significance vs. unbounded (McNemar, paired bootstrap 95% CI): ***p¡0.001, **p¡0.01.

Reading Table 8 with the *Mean gen-fail* column reveals the mechanism: as k shrinks, tasks failing to produce executable code rise (17.4 at k=10, 27.6 at k=3, 34.6 at k=1), driving Pass@1 downward. The budget is exhausted at the tool boundary, but the agent keeps running; with no further `run_code` calls it cannot finish or validate its generation. This $k\downarrow\rightarrow$ gen-fail$\uparrow\rightarrow$ Pass@1$\downarrow$ chain is the observable signature of Φ being bounded, consistent with Principle 3.

6.2.3 S3: The prior axis, the contrast arm of RQ1. Principle 1 separates knowledge from belief: the frozen prior should bound what a harness can achieve without substituting for observation. Table 9 changes only $\mathcal{K}$—the LLM—with the harness fixed. We use DeepSeek-V4-Pro (FP4) as the primary backbone for its cost–capability trade-off; GLM-5.2 attains marginally higher Pass@1 (83.3%) but overlaps within one standard deviation. The spread is what the $\mathcal{K}$ axis can move—and it is smaller than what S1's fidelity ladder and S2's effector ablation move, confirming Principle 2: the prior does not substitute for the observation channel.

Read against Principle 2, this spread is what $\mathcal{K}$ can move—smaller than the observation-channel effects in S1–S2, confirming that the prior does not substitute for observation.

6.2.4 RQ3: re-measuring the structure per toolchain. The three slices of **EDA-Scripting-Bench** share one task schema, one assertion criterion, and one harness, so S1–S2 can be re-run slice by slice with nothing changed but the toolchain. Table 10 reports the component ablation on SKILL (157 tasks) and Tcl (134 tasks, aggregating WaveView and Innovus), alongside the PyAether anchor; the *Overall* column pools all 449 tasks. The comparison of interest is the *within-language* ordering, not cross-table absolute rates.

Table 9: Impact of LLM backbone (ZhuLong)

LLM Backbone	Pass@1 (%)	Δ
DeepSeek-V4-Pro	81.4 ± 1.5	—
GLM-5.2	83.3 ± 3.1	+1.9 pp^{ns}
DeepSeek-V4-Flash†	60.1 ± 5.5	-21.3 pp***
Kimi-K2.6	77.0 ± 1.6	-4.4 pp**
Doubao-Seed-2.0-Pro	63.8 ± 1.4	-17.6 pp***

Significance vs. DeepSeek-V4-Pro (McNemar on 790 paired observations, paired bootstrap 95% CI): ***p¡0.001, **p¡0.01, ns = not significant. GLM-5.2: p=0.38, CI $[-2.7, +5.4]$ pp. Kimi-K2.6: p=0.003, CI $[+1.5, +8.4]$ pp. Doubao: $p=9\times10^{-23}$, CI $[+13.3, +23.3]$ pp. Flash†: retest 5-run under reduced gateway parallelism (-p 4) and enlarged token budget (max_tokens 16384); the initial 5-run (11.9 ± 2.7%, Δ= − 69.5 pp) was depressed by an infrastructure confound (API-gateway concurrency cap and a reasoning-token budget exhausted before content emission), not by model capability. McNemar p and CI on the retest set are not re-computed; the *** marker is inherited from the initial pooled analysis and should be read descriptively.

Table 10: Cross-language reproduction of the component ablation (RQ3). Pass@1 (%), 5-run mean±std on PyAether (158 tasks) and SKILL (157 tasks); Tcl aggregates the Custom WaveView (77) and Innovus (57) runtimes into a single 134-task column (task-weighted). The Overall column pools all 449 tasks into one task-weighted Pass@1 per configuration (single value, no run-level dispersion). The PyAether column is the main-table anchor (Table 4). The full system is the top configuration on every language; the prior-only baseline is the bottom on every language.

Configuration	PyAether	SKILL	Tcl	Overall
Pure LLM	10.5 ± 1.9	33.2 ± 2.3	10.2	18.3
RAG (retrieval only)	72.2 ± 1.6	33.9 ± 10.6	84.6	62.5
ZhuLong w/o Retrieval	81.0 ± 4.5	39.0 ± 12.6	73.2	64.0
ZhuLong (Full)	**81.4 ± 1.5**	**60.3 ± 6.7**	**87.2**	**75.8**

The qualitative structure reproduces across all three languages: the full system is the *top* configuration and the prior-only baseline the *bottom* on every language—the dominance of observation-plus-execution is domain-structural, not toolchain-specific (RQ3). The intermediate ordering is less stable: on PyAether the sandbox-only variant beats RAG, whereas on Tcl the reverse holds and on SKILL the two are close, reflecting that different toolchains expose different amounts of usable documentation relative to executable feedback. The two extremes are invariant, and the *Overall* column reproduces the same ordering at the pooled level. The (Ω) and (Φ) axes are reported on PyAether only.

Table 11: Failure classification for the representative trace (129/158 pass, 29 fail). Execution-stage failures are classified by the L1–L4 hierarchy of Section 4. Generation-stage failures are broken down by cause.

Category	Count	Example Tasks
Execution-stage failures (12)		
L1: Syntactic / API-name	0	—
L2: Parameter / signature	0	—
L3: Semantic / logic	4	017, 052, 058, 144
L4: Situated / design-state	8	010, 063, 068, 074, 109, 142, 146, 155
Generation-stage failures (17)		
Timeout (>2500 s)	4	105, 123, 124, 129
Wrong save path	10	104, 106, 118, 119, 128, 131, 145, 152, 158
No code produced	3	013, 072, 120

6.3 Error Analysis

To characterize failure modes, we inspected a representative trace from the full-system 5-run set—the run whose Pass@1 (81.6%, 129/158) is closest to the 5-run mean (81.4±1.5%, Table 4). The 29 failures comprise 17 generation-stage failures (no executable code produced) and 12 execution-stage failures (code ran but assertions failed). We classify the 12 execution-stage failures by the L1–L4 information hierarchy of Section 4: L1 (syntactic/API-name errors), L2 (parameter/signature errors), L3 (semantic/logic errors), and L4 (situated truthfulness violations—hidden type contracts, internal selection state, layout-dependent preconditions that no offline knowledge can resolve).

The dominant failure mode is L4: 8 of 12 execution-stage failures (67%) are situated truthfulness violations that no amount of offline knowledge can prevent—the code is syntactically correct (L1), uses documented APIs with plausible arguments (L2–L3), yet fails because it violates an undocumented binding contract, depends on an internal selection state, or assumes a runtime context that does not hold. This directly supports the grounding-gap framework: the correctness criterion is an L4 property, and execution is the only channel that can observe the current design state.

6.3.1 L4: Situated truthfulness violations. The agent selects the correct API family and the code executes, but the result violates an undocumented contract that the bindings enforce or the test oracle checks—hidden type expectations, internal state dependencies, or runtime-context preconditions invisible in documentation.

Case study: Task 010 (add_instance_to_schematic). The agent's code creates a schematic instance and its own `run_code` tests report "ALL ASSERTIONS PASSED," yet the official harness fails: `target_cv must be an opened cellview object.` The agent passed a handle valid in its own context but not satisfying the "opened" precondition—an L4 property of the live design state, not of the API signature (L3).

Tasks 063, 146, 074, 155 follow analogous patterns: correct API calls that return internal type names depending on which binding representation is exposed, attributes that exist in documentation but not in the test's execution path, self-invented wrappers for C++ types documentation doesn't show how to construct, and return values that depend on whether a GUI draw area is active—all situated properties observable only through execution.

6.3.2 L3: Semantic and logic errors. The agent selects valid APIs and the code executes without runtime errors, but the composition logic—parameter propagation, post-processing, or control flow—fails the oracle. *Task 052*: the agent replaced the mandated `EmyNtlUtil.checkViewList` call with a custom Python validator that returned an error on the harness's configuration. *Task 058*: substring matching (`"high" in queue_name`) was used instead of the required equality check. Tasks 017 (instance-counting returns 0) and 144 (property set to `None`) follow the same pattern: valid API calls composed with incorrect logic.

6.3.3 Generation-stage failures. The 17 generation-stage failures fall into three categories (Table 11): *timeout* (4 tasks exceeding 2500 s on complex multi-step tasks), *wrong save path* (10 tasks saving to `ws/` instead of `ws/pyAether_code/`), and *no code produced* (3 tasks). The wrong-save-path failures are notable: the code is likely correct but lost the point on a procedural (L1) error, amenable to harness-level mitigation rather than knowledge improvement.

6.3.4 Implications. The L1–L4 classification makes the grounding-gap framework concrete: 67% of execution-stage failures are L4 (situated truthfulness), 33% are L3, and 0% are L1/L2. No offline knowledge can prevent L4 failures—the violated contracts are invisible until the agent acts on the live design and reads back the result. This is why the execution loop is decisive: the sandbox is the only channel through which the agent can observe hidden type contracts, internal selection states, and runtime-context dependencies. These findings motivate type-aware API documentation and case-level knowledge distillation for agent self-evolution.

7 Discussion

The error analysis in Section 6.3 shows that the 12 execution-stage failures mainly arise from API misuse and incorrect API composition. These failures are largely caused by undocumented API contracts, missing usage-level knowledge, and insufficient validation feedback, motivating type-aware retrieval and more informative execution feedback.

Beyond these specific improvements, our results carry broader implications. The dominance of sandbox execution (+9.2 pp) over static knowledge suggests that for domains with long-tailed, under-documented APIs, execution grounding is not merely beneficial but essential. Our theoretical characterization (Section 4) elevates this from an empirical observation to a structural principle: the grounding gap is bounded by observation fidelity, so execution channels—not larger priors—are the binding constraint in such domains.

This paradigm may generalize to other domain-specific scripting tasks with similar characteristics.

Limitations

Several limitations warrant acknowledgment. First, performance varies significantly across base LLMs (60.1%–83.3%), indicating that the underlying model remains a decisive factor. Second, the interactive bridge is single-threaded, limiting concurrent deployment. Third, while realistic, the main results are measured on the 158-task PyAether slice; the SKILL (158) and Tcl (133) slices are complete and a cross-language reproduction of the component ablation is reported (Section 6.2.4), but the fidelity (Ω) and belief-update (Φ) axes are reported on PyAether only, so every figure outside the RQ3 table reflects the PyAether slice. Fourth, although each configuration was evaluated over five independent runs, the benchmark size limits statistical power to certify small effects; we therefore report McNemar and paired-bootstrap tests for the main contrasts (Table 4), but treat small gaps such as the full vs. w/o-retrieval difference (-0.4 pp, p=0.81) and GLM-5.2 vs. the primary backbone ($+1.9$ pp) as directional rather than certified. Fifth, the primary backbone is a quantized deployment (DeepSeek-V4-Pro, FP4), and we do not separately ablate quantization level, so the effect of quantization on scripting accuracy is left to future work. Sixth, the convergence-cost comparison is complete only for (F) and (B): the legacy (N$'$) traces were discarded in an earlier storage migration, so its cost is reported as not recoverable rather than imputed.

8 Conclusion and Future Work

This paper presented **ZhuLong**, an execution-grounded LLM coding agent for PyAether, SKILL, and Tcl that closes the loop from generation to execution to iterative refinement via sandbox execution feedback. We further provided a **theoretical grounding**: EDA correctness is a situated (L4) property, and a Bayesian analysis shows the resulting grounding gap is bounded by observation fidelity—not model scale—explaining why runtime observation is necessary. Evaluated on **EDA-Eval-PyAether**—the first benchmark for PyAether scripting, comprising 158 real-world tasks—the complete system achieves 81.4% Pass@1 ($\pm$1.5, 5-run mean), substantially outperforming static RAG (72.2%). Ablation studies show that execution feedback and API retrieval are jointly necessary. These results demonstrate that, for domain-specific scripting with long-tail, under-documented APIs, execution grounding is not merely beneficial but essential.

Beyond the specific results, this work establishes three foundations for future research: (1) a **theoretical characterization**—the grounding gap—that diagnoses why situated (L4) correctness demands runtime observation and bounds the gap by observation fidelity; (2) a **methodological paradigm**—execution-grounded agents—that can generalize to other domains with similar characteristics (long-tail APIs, incomplete documentation, available execution sandboxes); and (3) an **evaluation infrastructure**—**EDA-Scripting-Bench** (PyAether + SKILL + Tcl, 449 tasks), whose PyAether slice EDA-Eval-PyAether anchors the main results and whose SKILL and Tcl slices support a cross-language reproduction (Section 6.2.4)—that enables systematic cross-language comparison and progress tracking for LLM-based EDA scripting. The harness, task schema and evaluation protocol will be released; the task cases themselves derive from vendor documentation and customer designs and cannot be redistributed.

Future work proceeds along three directions. First, extending all ablation axes to the full EDA-Scripting-Bench (PyAether + SKILL + Tcl, 449 tasks), together with extending the benchmark to systematically cover GUI-bridged tasks on open design sessions. Second, evolving the agent through self-improvement—learning from both successful and failed execution traces to refine its retrieval, code generation, and exploration strategies. Third, adapting the architecture to additional EDA platforms, including Synopsys tools with Tcl scripting, to broaden the scope of execution-grounded EDA agents.

References

[1] Anthropic. 2024. Model Context Protocol (MCP). https://modelcontextprotocol.io/.

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).

[3] Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. 2024. Can EDA Tool Feedback Improve Verilog Generation by LLMs? *arXiv preprint arXiv:2411.11856* (2024).

[4] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv:2402.03216 [cs.CL] https://arxiv.org/abs/2402.03216

[5] Lei Chen, Yiqi Chen, Zhufei Chu, Wenji Fang, Tsung-Yi Ho, Yu Huang, Sadaf Khan, Min Li, Xingquan Li, Yun Liang, Yibo Lin, Jinwei Liu, Yi Liu, Guojie Luo, Zhengyuan Shi, Guangyu Sun, Dimitrios Tsaras, Runsheng Wang, Ziyi Wang, Xinming Wei, Zhiyao Xie, Qiang Xu, Chenhao Xue, Evangeline F Y Young, Bei Yu, Minjie Yuan, Haoyi Zhang, Zuodong Zhang, Yuxiang Zhao, Hui-Ling Zhen, Ziyang Zheng, Binwu Zhu, Keren Zhu, and Sunan Zou. 2024. The Dawn of AI-Native EDA: Promises and Challenges of Large Circuit Models. *arXiv preprint arXiv:2403.07257* (2024). https://arxiv.org/abs/2403.07257

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[7] Cline Contributors. 2024. Cline: Autonomous AI Coding Assistant. https://github.com/cline/cline.

[8] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). arXiv:2401.08281 [cs.LG]

[9] Empyrean Technology. 2026. Empyrean PyAether: Full Custom Design Platform Ecosystem. https://empyrean.com.cn/solutions/index-128.html. Accessed: 2026-06-28.

[10] Xue Jiang, Yihong Dong, Zhi Jin, and Ge Li. 2024. SEED: Customize Large Language Models with Sample-Efficient Adaptation for Code Generation. *arXiv preprint arXiv:2403.00046* (2024).

[11] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. 1998. Planning and acting in partially observable stochastic domains. *Artificial Intelligence* 101, 1-2 (1998), 99–134.

[12] Rajesh Kumar et al. 2026. AgentForge: Execution-Grounded Multi-Agent LLM Framework for Autonomous Software Engineering. *arXiv preprint arXiv:2604.13120* (2026). https://arxiv.org/abs/2604.13120

[13] Qiufeng Li, Rongqian Chen, Quan Cheng, Chengxuan Wang, Sizhe Tang, Wuxi Li, Duo Ding, Chia-Tung Ho, Haoxing Ren, David Z. Pan, Tian Lan, and Weidong Cao. 2026. PDAGENT-BENCH: Characterizing, Grounding, and Architecting LLM Agents for VLSI Physical Design. *arXiv preprint arXiv:2606.17253* (2026).

[14] Bingyang Liu, Haoyi Zhang, Xiaohan Gao, Zichen Kong, Xiyuan Tang, Yibo Lin, Runsheng Wang, and Ru Huang. 2025. LayoutCopilot: An LLM-Powered Multiagent Collaborative Framework for Interactive Analog Layout Design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 44, 8 (2025), 3126–3139. doi:10.1109/TCAD.2025.3529805

[15] Shang Liu, Wenji Fang, Yao Lu, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2024. RTLCoder: Outperforming GPT-3.5 in Design RTL Generation with Our Open-Source Dataset and Lightweight Solution. In *2024 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, New York, NY, USA. Also available at arXiv:2312.08617. https://arxiv.org/abs/2312.08617

[16] Yiyi Lu, Hoi Ian Au, Junyao Zhang, Jingyu Pan, Yiting Wang, Ang Li, Jianyi Zhang, and Yiran Chen. 2025. AutoEDA: Enabling EDA Flow Automation through Microservice-Based LLM Agents. *arXiv preprint arXiv:2508.01012* (2025). Uses MCP protocol for EDA automation. https://arxiv.org/abs/2508.01012

[17] OpenAI. 2023. *GPT-4 Technical Report*. Technical Report arXiv:2303.08774. OpenAI.

[18] Yuan Pu, Zhuolun He, Tairu Qiu, Haoyuan Wu, and Bei Yu. 2024. Customized Retrieval Augmented Generation and Benchmarking for EDA Tool Documentation QA. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, New York, NY, USA. arXiv:2407.15353 [cs.CL]

[19] Yibo Rui, Yuanhang Li, Rui Wang, Ruiqi Chen, Yanxiang Zhu, Zhixiong Di, Xi Wang, and Ming Ling. 2025. ChaTCL: LLM-Based Multi-Agent RAG Framework for TCL Script Generation. In *Proceedings of the International Symposium of Electronics Design Automation (ISEDA)*. IEEE, New York, NY, USA, 736–742.

[20] Cadence Design Systems. 2024. *Virtuoso Studio SKILL Language Reference*. https://www.cadence.com

[21] Shailja Thakur, Baleegh Ahmad, Hammond A Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2023. VeriGen: A Large Language Model for Verilog Code Generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2023), 1–31.

[22] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. 2005. *Probabilistic Robotics*. MIT Press.

[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. *Advances in Neural Information Processing Systems* 30 (2017), 5998–6008.

[24] Ning Xu, Zhaoyang Zhang, Senlin Shu, Qi Lei, Jiaqi Lv, Wensuo Wang, Tianhao Zhao, Chao Zhang, Zhaoliang Yang, Xiangyu Li, Zhaorui Su, Jingshan Li, and Xin Geng. 2026. iScript: A Domain-Adapted Large Language Model and Benchmark for Physical Design Tcl Script Generation. *arXiv preprint arXiv:2603.04476* (2026).

[25] An Yang et al. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.